# A thirty-three-variable consistency equation

Positive integer unknowns; exact degree 8; a 139-operation evaluator.

The only unknowns are the thirty-three distinct positive integers

$$J, F, \alpha, f, g, T, X, Y, \xi, \eta, \zeta, \upsilon, \gamma, \vartheta, \psi, \varphi, \epsilon,$$
$$c, a, \ell, h, t, s, x, m, n, p, j, d, o, e, z, \delta \in \mathbb{Z}_{>0}.$$

All other symbols below are polynomial abbreviations:

$$b = \beta J, \quad q = 1 + b, \quad C = q - F - \alpha, \quad U = Yq^2,$$
$$E = XY, \quad k = \eta + \zeta, \quad W = h - t,$$
$$H = 3 + 4a, \quad \Delta = \delta, \quad L = X + ac,$$
$$M = sb,$$
$$R = [\alpha + (q - F)b](q^2 - 1) + J(\mu + \omega q),$$
$$Z = p - j, \quad D = d - o, \quad V = e - z,$$
$$I = ng, \quad Q = \Delta(m - 1),$$
$$\mathcal{P} = Z^2 - \psi^2[1 + \ell(\ell + k)],$$
$$\mathcal{D} = D^2 - \varphi^2(1 + \Delta n),$$
$$\mathcal{Q} = \upsilon^2 - Q[\upsilon^2 - \vartheta] - 1,$$
$$\mathcal{R} = I^2 - Q, \quad \mathcal{E} = \vartheta - V^2,$$
$$\mathcal{B} = \psi - M, \quad \mathcal{C} = \varphi - U, \quad \mathcal{G} = \epsilon - R,$$
$$\mathcal{H} = c - \eta - Yk, \quad \mathcal{T} = a - E - Y,$$
$$\mathcal{L} = \ell - EYk, \quad \mathcal{W} = W - [C(X + Kq) - Fq],$$
$$\mathcal{X} = s - q^3, \quad \mathcal{Y} = x - Eq,$$
$$\mathcal{Z} = m - f^2, \quad \mathcal{N} = n - c^2,$$
$$\mathcal{I} = Z - [\psi(\ell + k + \xi - kW) + Yb - Ws],$$
$$\mathcal{J} = D - [\varphi L + H(\varphi\gamma + x + \epsilon + 1 - k)],$$
$$\mathcal{K} = V - [m(Tc - \epsilon) - c], \quad \mathcal{O} = \delta - H - a^2.$$

The single Diophantine equation is

$$\mathcal{P}^2 + \mathcal{D}^2 + \mathcal{Q}^2 + \mathcal{R}^2 + \mathcal{E}^2 + \mathcal{B}^2 + \mathcal{C}^2 + \mathcal{G}^2$$
$$+\mathcal{H}^2 + \mathcal{T}^2 + \mathcal{L}^2 + \mathcal{W}^2 + \mathcal{X}^2 + \mathcal{Y}^2 + \mathcal{Z}^2 + \mathcal{N}^2$$
$$+\mathcal{I}^2 + \mathcal{J}^2 + \mathcal{K}^2 + \mathcal{O}^2 = 0.$$

**The four fixed integers.** Every index, set, sum, product and floor below defines a constant, not another unknown. Put

$$\rho = 2^{47}, \quad B = \rho^{2011}, \quad \beta = B - 1,$$
$$G_m(x) = \sum_{j=0}^{m-1} x^j, \quad \Lambda = \rho^{1005}.$$

$$\mu = \beta - G_{951}(\rho) - \rho^{960} G_{26}(\rho) - \rho^{1004},$$
$$\omega = \beta + 8G_{917}(\rho) - 4 + (2^{45} - 4)\Lambda + \rho^{960} G_9(\rho) + \rho^{987} G_8(\rho) + \rho^{1004},$$
$$K = \rho^{18} + \rho^{1004} + \rho^{2020} + 2\Lambda\Phi(1/\rho),$$

where

$$\chi_i = 1 + 2^{10}(1 + w_i + 2^8 n_i + 2^{17} p_i + 2^{25} s_i) \quad (0 \le i \le 941),$$
$$\pi(0) = 0, \quad \pi(i) = 1 + \sum_{j=1}^{941} \left\lfloor \frac{2^{44} - 1 + \chi_j - \chi_i}{2^{44}} \right\rfloor \quad (1 \le i \le 941).$$

$$\Phi(x) = \sum_{i=0}^{941} \chi_i x^{\pi(i)} + 2^{10} x^{969}(x^9 + 2^{17}) G_8(2x) + 2^{18} x^{942}(x^{18} + 2^{17}) G_9(2x).$$

The rational intermediate in K cancels exactly; all four results are positive integers.

**The 942 indexed fixed tiles.** Use the actions $u_j, v_j, d_j$ decoded below. For $d = 0, 1$, let

$$h(q, b) = 2q + b + 2, \quad \mathcal{S}_d = \{u_j : d_j = d,\ 0 \le j < 430\},$$
$$r_{d,q} = 205 - 199d + |\mathcal{S}_d \cap \{0, \dots, q - 1\}|.$$

An interval ending at $-1$ is empty. The table defines $t_i$ for $0 \le i \le 941$; all index ranges are fixed:

| index $i$ | tile $t_i = (w_i, n_i, p_i, s_i)$ |
|---|---|
| 0 | $(2, 435, 3, 1)$ |
| $j + 1$ $(j = 2q + b < 430)$ | $((1 - d_j) r_{0,u_j}, h(q, b), d_j r_{1,u_j}, v_j)$ |
| $433 - 2a + b$ $(a, b = 0, 1)$ | $(0, 432a + b, 0, 432a + b)$ |
| $\substack{423 + 2r_{d,q} + b \\ q \in \mathcal{S}_d,\ d,b = 0,1}$ | $(d r_{d,q}, b, (1 - d) r_{d,q}, h(q, b))$ |
| $923 + 2a + b$ $(a, b = 0, 1)$ | $(0, 432a + b, 0, 438)$ |
| 927 | $(2, 434, 2, 0)$ |
| 928 | $(3, 436, 4, 4)$ |
| 929 | $(4, 436, 4, 0)$ |
| $496 + t$ $(t = 434, 435, 436)$ | $(1, 438, 1, t)$ |
| $933 + 3a + b$ $(a, b = 0, 1, 2)$ | $(a^2, 437, b, 437)$ |

**A closed decoder for the 430 actions.** Put $D_b(A, k) = \left\lfloor A/b^k \right\rfloor \bmod b$, where $a \bmod b = a - b\lfloor a/b \rfloor$. For $0 \le q < 215$, $b = 0, 1$, $j = 2q + b$:

$$\mathcal{N} = \mathcal{A} \bmod 2^{636}, \quad \mathcal{T} = \left\lfloor \mathcal{A}/2^{636} \right\rfloor,$$
$$\nu_j = D_2(\mathcal{N}, j), \quad \kappa_j = (\sigma_j + 2)^{1 - \nu_j} \quad (0 \le j < 430),$$
$$\sigma_j = \sum_{k=0}^{j-1} \nu_k, \quad \Pi_j = \prod_{k=0}^{j-1} \kappa_k \quad (0 \le j \le 430),$$
$$m_q = D_2(\mathcal{N}, 430 + q), \quad \tau_q = \sum_{k=0}^{q-1} m_k,$$
$$F_q = 14 + m_q(D_{16}(\lfloor \mathcal{T}/\Pi_{430} \rfloor, \tau_q) - 14),$$
$$u_j = (\sigma_{j+1} - (\lfloor \mathcal{T}/\Pi_j \rfloor \bmod \kappa_j)) \bmod 216,$$
$$v_j = D_2(F_q, 2b), \quad d_j = D_2(F_q, 2b + 1).$$

Empty sums/products are 0/1. Here $(v_0, d_0, u_0) = (1, 1, 1)$; the constructor tile uses this actual first target, including its state renaming.

**The integer $\mathcal{A}$ in base 10.** Its complete decimal expansion is the concatenation of these lines:

```
12889260410359561746736540743515845134244260640012702033212621
95398074180784058571518488282119861023093355389406939414042892
97455773992193784795705001618777025966369137954859764532832722
75411843410809038713502633204617333246733616940964578272360901
56186216508504953649192752154519268024509952082950238865468307
22999632482769253980274091217460707829023970817244790340704449
09085581271192071656170732004937337082171141082486384269072884
72778087419663417942892520175630649792329265834490253035577533
96449689808375788457488971017043689875433162972390177936123032
89025657757489526342497438772045714099661532481123050235898973
27443609903800541199351405985754278307227777584454508422717998
33411356440933534607489268032843 7
```

The fixed integer $\mathcal{A}$ above is written in base 10. Lean proves that the formalized equation has no positive solution exactly when the chosen arithmetized ZFC consistency statement holds, by a derivation in the specified intuitionistic PRA. The literal endpoint was independently replayed on a fixed source snapshot; some transitive dependencies remain inherited. The exact degree is certified by integer-polynomial calculations; the 139-operation count is for the unshifted circuit. Neither count is a minimality claim.

# Exact 139-operation circuit

Each numbered line defines one gate over the integers. The inputs are the thirty-three unknowns and the four fixed integers above. The ASCII names `alpha, xi, eta, zeta, upsilon, gamma, vartheta, psi, varphi, epsilon, beta, mu, omega, ell, Delta, delta` refer to their displayed counterparts; `g` is the unknown $g$. The last gate is zero exactly when the displayed equation holds. The circuit inputs `cNamed, aNamed, ellNamed` are $c, a, \ell$; `WNamed` is the displayed $W$, while gate `W` is $C(X + Kq) - Fq$. The inputs `s, x, m, n, p, j, d, o, e, z, delta` are the displayed $s, x, m, n, p, j, d, o, e, z, \delta$. Intermediate gate names below are local abbreviations, not additional unknowns.

1. `b = J * beta`
2. `q = 1 + b`
3. `q2 = q * q`
4. `q3 = q * q2`
5. `U = Y * q2`
6. `qF = q - F`
7. `C = qF - alpha`
8. `E = X * Y`
9. `k = eta + zeta`
10. `kY = Y * k`
11. `c = eta + kY`
12. `c2 = cNamed * cNamed`
13. `a = E + Y`
14. `g15 = 4 * aNamed`
15. `H = 3 + g15`
16. `g17 = aNamed * aNamed`
17. `DeltaComputed = H + g17`
18. `g19 = aNamed * cNamed`
19. `L = X + g19`
20. `ell = E * kY`
21. `M = s * b`
22. `g25 = K * q`
23. `g26 = X + g25`
24. `g27 = C * g26`
25. `g28 = F * q`
26. `W = g27 - g28`
27. `g30 = ellNamed + k`
28. `g31 = g30 + xi`
29. `WNamed = h - t`
30. `root_skew = k * WNamed`
31. `root_inner = g31 - root_skew`
32. `root_scaled = psi * root_inner`
33. `g33 = Y * b`
34. `root_btq3 = WNamed * s`
35. `root_sum = root_scaled + g33`
36. `ZComputed = root_sum - root_btq3`
37. `Z = p - j`
38. `g38 = Z * Z`
39. `g39 = psi * psi`
40. `g40 = ellNamed * g30`
41. `g41 = 1 + g40`
42. `g42 = g39 * g41`
43. `A0 = g38 - g42`
44. `g44 = qF * b`
45. `rankseed = alpha + g44`
46. `g46 = q2 - 1`
47. `g47 = rankseed * g46`
48. `g48 = omega * q`
49. `g49 = mu + g48`
50. `g50 = J * g49`
51. `R = g47 + g50`
52. `g52 = k - epsilon`
53. `r0 = g52 - 1`
54. `g54 = varphi * gamma`
55. `g55 = E * q`
56. `g56 = g54 + x`
57. `shift = g56 - r0`
58. `g58 = varphi * L`
59. `g59 = H * shift`
60. `DstarComputed = g58 + g59`
61. `D = d - o`
62. `g61 = D * D`
63. `g62 = varphi * varphi`
64. `g63 = delta * n`
65. `g64 = 1 + g63`
66. `g65 = g62 * g64`
67. `A1 = g61 - g65`
68. `f2 = f * f`
69. `rho = m - 1`
70. `Q = delta * rho`
71. `I = n * g`
72. `g71 = I * I`
73. `g72 = g71 - Q`
74. `g74 = T * cNamed`
75. `g75 = g74 - epsilon`
76. `g76 = m * g75`
77. `VComputed = g76 - cNamed`
78. `V = e - z`
79. `y2 = upsilon * upsilon`
80. `g80 = V * V`
81. `g81 = y2 - vartheta`
82. `g82 = Q * g81`
83. `Fa = y2 - g82`
84. `auxResidual = Fa - 1`
85. `firstSquare = A0 * A0`
86. `mainSquare = A1 * A1`
87. `auxSquare = auxResidual * auxResidual`
88. `strongSquare = g72 * g72`
89. `firstMain = firstSquare + mainSquare`
90. `firstMainAux = firstMain + auxSquare`
91. `sumSquares = firstMainAux + strongSquare`
92. `difference = vartheta - g80`
93. `constraint = difference * difference`
94. `result = sumSquares + constraint`
95. `scaleDiff = psi - M`
96. `scaleCheck = scaleDiff * scaleDiff`
97. `final = result + scaleCheck`
98. `mainDiff = varphi - U`
99. `mainCheck = mainDiff * mainDiff`
100. `result16 = final + mainCheck`
101. `rankDiff = epsilon - R`
102. `rankCheck = rankDiff * rankDiff`
103. `result17 = result16 + rankCheck`
104. `cDifference = cNamed - c`
105. `cConstraint = cDifference * cDifference`
106. `aDifference = aNamed - a`
107. `aConstraint = aDifference * aDifference`
108. `ellDifference = ellNamed - ell`
109. `ellConstraint = ellDifference * ellDifference`
110. `wDifference = WNamed - W`
111. `wConstraint = wDifference * wDifference`
112. `addC = result17 + cConstraint`
113. `addA = addC + aConstraint`
114. `addEll = addA + ellConstraint`
115. `finalWithNamedValues = addEll + wConstraint`
116. `cubeDifference = s - q3`
117. `cubeConstraint = cubeDifference * cubeDifference`
118. `productDifference = x - g55`
119. `productConstraint = productDifference * productDifference`
120. `fSquareDifference = m - f2`
121. `fSquareConstraint = fSquareDifference * fSquareDifference`
122. `cSquareDifference = n - c2`
123. `cSquareConstraint = cSquareDifference * cSquareDifference`
124. `addCube = finalWithNamedValues + cubeConstraint`
125. `addProduct = addCube + productConstraint`
126. `addFSquare = addProduct + fSquareConstraint`
127. `addCSquare = addFSquare + cSquareConstraint`
128. `zDifference = Z - ZComputed`
129. `zConstraint = zDifference * zDifference`
130. `dDifference = D - DstarComputed`
131. `dConstraint = dDifference * dDifference`
132. `vDifference = V - VComputed`
133. `vConstraint = vDifference * vDifference`
134. `deltaDifference = delta - DeltaComputed`
135. `deltaConstraint = deltaDifference * deltaDifference`
136. `addZConstraint = addCSquare + zConstraint`
137. `addDConstraint = addZConstraint + dConstraint`
138. `addVConstraint = addDConstraint + vConstraint`
139. `addDELTAConstraint = addVConstraint + deltaConstraint`